

Universidad Tecnológica del Perú

Facultad de Ingeniería de Sistemas e Informática

Construcción de una Aplicación Web Empresarial

Avance de Trabajo Final 3: Plataforma E-commerce de Indumentaria Deportiva "LINKSPORT"

Díaz Malca, Jhon Franklin

Doza Rudas, Jean Franco

Sotomayor, José Luis

Marcos de Desarrollo Web

Docente: Bernardo Climaco Hermitaño Atencio

Ciclo 2026-II

Lima, Perú

2026

CAPÍTULO 1: ASPECTOS GENERALES

1.1. Definición del problema

El comercio electrónico peruano ha mantenido un crecimiento sostenido y la indumentaria deportiva figura entre las categorías de mayor interés para el consumidor digital (CAPECE, 2023). En este contexto, las micro y pequeñas empresas (MYPEs) necesitan plataformas que les permitan actualizar su oferta y registrar sus ventas sin depender de la edición manual del código fuente.

En el Avance 2, LINKSPORT migró de un catálogo web estático a una aplicación Spring Boot con operaciones CRUD y vistas dinámicas. Sin embargo, esa versión almacenaba los datos en colecciones ArrayList en memoria, lo que generó las siguientes limitaciones:

- **Pérdida de información:** los productos y usuarios registrados desaparecían al reiniciar la aplicación.
- **Ausencia de relaciones:** los productos no estaban asociados a categorías ni existía un registro de las compras realizadas por los clientes.
- **Datos sin control de calidad:** el sistema podía aceptar nombres vacíos, precios negativos o productos sin imagen, pues no existían reglas de validación.
- **Carrito sin trazabilidad:** el carrito calculaba el subtotal en el navegador, pero la compra no quedaba registrada en ningún lugar.

Para que LINKSPORT opere como una tienda en línea confiable, requiere una capa de persistencia relacional, validación de los datos de entrada y registro de ventas. Este avance responde a esa necesidad.

1.2. Definición de objetivos

1.2.1. Objetivo general

Desarrollar el frontend y la lógica de backend de la aplicación web de LINKSPORT, incorporando persistencia en una base de datos relacional PostgreSQL mediante JPA e Hibernate, validación de datos con Spring Validation y una interfaz responsiva construida con el framework CSS Bulma.

1.2.2. Objetivos específicos

1. Modelar cinco entidades JPA (Producto, Categoría, Usuario, Venta y DetalleVenta) y establecer las relaciones uno a muchos entre ellas.
2. Reemplazar el almacenamiento en memoria del catálogo por repositorios Spring Data JPA conectados a PostgreSQL.

3. Implementar las operaciones de ingresar, mostrar, buscar, editar y eliminar productos, tanto desde servicios REST como desde un panel web con Thymeleaf.
4. Aplicar Spring Validation en las clases del modelo y activarlo en los controladores, mostrando los mensajes de error en las vistas.
5. Registrar las compras del carrito como una venta con sus detalles en la base de datos.
6. Diseñar vistas responsivas con HTML5 semántico y Bulma, incluyendo catálogo por categorías, carrito lateral y ventanas modales de acceso.

CAPÍTULO 2: MARCO TEÓRICO

2.1. Antecedentes

En el Avance 2 las reglas de negocio de LINKSPORT se validaron con colecciones en memoria, una etapa habitual al prototipar aplicaciones web antes de integrar una base de datos (Walls, 2022). La arquitectura por capas (modelo, servicio y controlador) se diseñó precisamente para que ese almacenamiento pudiera reemplazarse sin alterar la capa de presentación, principio que se aplica en este avance.

Las tiendas en línea de mayor escala separan la administración del catálogo de la tienda pública y registran cada compra como una cabecera con sus líneas de detalle. Esta estructura se apoya en patrones de mapeo relacional, como el mapeo por clave foránea, que permiten recuperar el historial de ventas y relacionar cada línea con el producto vendido (Fowler, 2002). LINKSPORT adopta este esquema a escala MYPE: una venta pertenece a un cliente y contiene varios detalles, cada uno asociado a un producto.

Finalmente, el crecimiento del acceso móvil al comercio electrónico en el país (CAPECE, 2023) justifica mantener una interfaz responsiva (Marcotte, 2011) y validar los datos antes de guardarlos, de modo que la información del catálogo sea consistente.

2.2. Fundamento teórico

Mapeo objeto-relacional (ORM). Técnica que permite representar las tablas de una base de datos relacional como clases del lenguaje orientado a objetos, de modo que el programador manipula objetos y el ORM genera las sentencias SQL correspondientes (Bauer et al., 2015).

JPA e Hibernate. JPA (Jakarta Persistence API) es la especificación estándar de Java para el ORM y define anotaciones como @Entity, @Table, @Id y @Column. Hibernate es su implementación más difundida: traduce las operaciones sobre entidades a SQL según el dialecto de la base de datos y puede generar el esquema de tablas automáticamente (Bauer et al., 2015; Hibernate, s. f.).

Spring Data JPA. Módulo de Spring que simplifica la capa de acceso a datos. Al declarar una interfaz que extiende JpaRepository, Spring genera en tiempo de ejecución la implementación con los métodos del CRUD (save, findAll, findById, deleteById, entre otros) y permite definir consultas derivadas, como findByCorreo (Spring, s. f.-b).

PostgreSQL. Sistema gestor de bases de datos relacional de código abierto, que cumple con el estándar SQL y soporta integridad referencial mediante claves foráneas (PostgreSQL Global Development Group, s. f.). pgAdmin es su herramienta gráfica de administración, utilizada para crear la base de datos y verificar las tablas generadas.

Relaciones entre entidades. En JPA, la anotación @ManyToOne se ubica en el lado “muchos” y crea la clave foránea, mientras que @OneToMany con el atributo mappedBy representa el lado inverso. El atributo cascade propaga operaciones a las entidades hijas, orphanRemoval elimina los hijos que dejan de pertenecer a la colección y fetch = LAZY difiere la carga de la asociación hasta que se necesita (Bauer et al., 2015).

Bean Validation y Spring Validation. Jakarta Bean Validation es el estándar que permite declarar restricciones sobre los atributos mediante anotaciones, por ejemplo @NotBlank, @Size, @Positive, @DecimalMin y @Email (Jakarta EE, s. f.). Spring Boot lo integra con la dependencia spring-boot-starter-validation: al anotar un parámetro del controlador con @Valid, Spring valida el objeto antes de ejecutar la lógica y registra los errores, que pueden consultarse mediante un objeto BindingResult (Spring, s. f.-a).

Thymeleaf. Motor de plantillas que procesa el HTML en el servidor. Además de th:each y th:text, ofrece atributos para formularios (th:field) y para mostrar errores de validación (#fields.hasErrors y th:errors), lo que permite devolver al usuario el formulario con los mensajes correspondientes (Thymeleaf, 2025).

Arquitectura MVC y capa de servicio. El patrón Modelo-Vista-Controlador separa los datos, la presentación y el control de las peticiones (Gamma et al., 1994). En aplicaciones empresariales se añade una capa de servicio con la lógica de negocio y una capa de repositorio para el acceso a datos (Fowler, 2002).

API REST. Estilo de arquitectura en el que los recursos se manipulan con los métodos HTTP GET, POST, PUT y DELETE, equivalentes a consultar, crear, actualizar y eliminar.

Bulma y diseño responsivo. Bulma es un framework CSS de código abierto basado en Flexbox, con sistema de columnas, barra de navegación, tablas, formularios y componentes como card y modal, sin JavaScript propio (Bulma, 2025). Se utilizó para lograr un diseño que se adapta al tamaño de la pantalla (Marcotte, 2011).

CAPÍTULO 3: LA SOLUCIÓN

3.1. Descripción del sistema

El sistema web de **LINKSPORT** es una aplicación de comercio electrónico de indumentaria deportiva compuesta por dos componentes: la **tienda virtual**, orientada al cliente, y el **panel de administración de productos**, orientado al control del inventario. Ambos obtienen sus datos de una base de datos PostgreSQL.

La tienda virtual (ruta /) presenta un carrusel promocional, la tabla de servicios de entrega, la sección “Lo más vendido” (productos marcados como destacados) y las colecciones Running, Fútbol y Accesorios, que se generan dinámicamente según la categoría de cada producto. Incluye además un carrito lateral que calcula el subtotal en tiempo real y, al proceder al pago, registra la compra en la base de datos. El panel (ruta /ver-productos) permite registrar, editar y eliminar productos con validación de los datos ingresados.

Figura 1

Página principal de la tienda LINKSPORT con la tabla de servicios de entrega.

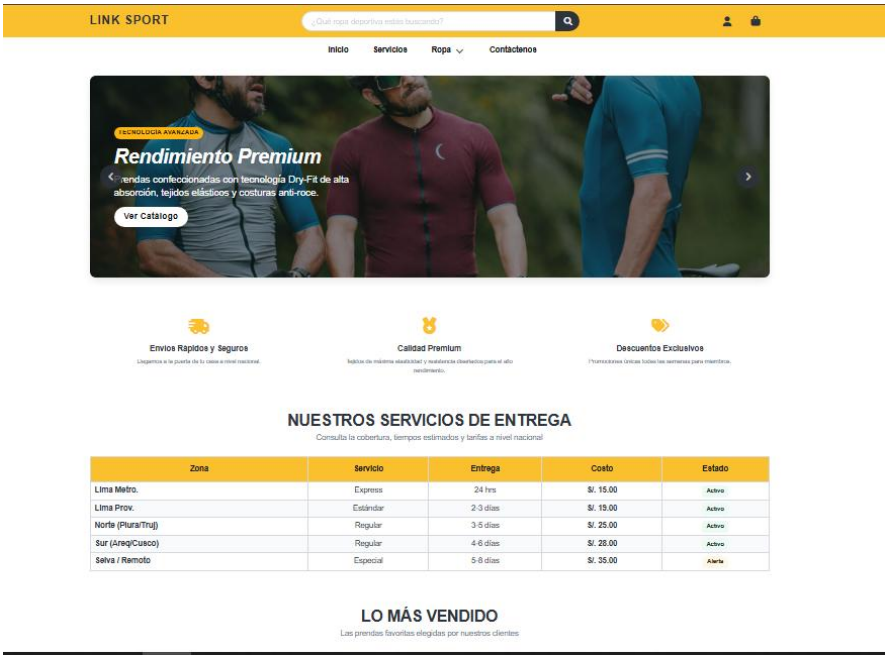


Tabla 1

Módulos y rutas del sistema.

Módulo	Ruta	Función
Tienda virtual	/	Catálogo dinámico por categorías, carrusel, carrito lateral y modales de acceso.
Panel de productos	/ver-productos	Formulario y tabla para registrar, editar y eliminar productos.

Módulo	Ruta	Función
Guardado desde formulario	/admin/productos/guardar (POST)	Valida el producto y lo guarda; si hay errores, vuelve al formulario.
API de productos	/api/productos	Servicios REST GET, POST, PUT y DELETE.
Registro de ventas	/api/productos/procesar-pago	Recibe el carrito y registra la venta con sus detalles.
API de usuarios	/api/usuarios	Listado y registro de usuarios.

3.2. Diseño de la arquitectura

La aplicación mantiene la arquitectura multicapa del Avance 2 y añade la capa de repositorio, que aísla el acceso a PostgreSQL. Cada capa se comunica con la siguiente mediante inyección de dependencias por constructor.

Tabla 2

Capas de la arquitectura y su responsabilidad.

Capa	Componentes	Responsabilidad
Vista	index.html, admin-productos.html, admin-usuarios.html (Thymeleaf + Bulma)	Presentar la información, capturar la interacción y mostrar los errores de validación.
Controlador	HomeController, ProductoController, UsuarioController	Recibir las peticiones HTTP, activar la validación, invocar los servicios y devolver una vista o JSON.
Servicio	ProductoService, UsuarioService	Aplicar la lógica de negocio, incluido el registro de ventas.
Repositorio	ProductoRepository, CategoriaRepository, UsuarioRepository, VentaRepository	Acceder a PostgreSQL mediante Spring Data JPA.
Modelo	Producto, Categoria, Usuario, Venta, DetalleVenta	Representar las entidades y sus restricciones de validación.

La estructura de paquetes del proyecto se muestra a continuación.

Código 1

Estructura de paquetes del proyecto linksport-backend.

```
linksport-backend/
├── pom.xml
└── src/main/
    ├── java/com/linksport/linksport_backend/
    │   ├── LinksportBackendApplication.java
    │   ├── HomeController.java
    │   ├── controller/  ProductoController, UsuarioController
    │   ├── service/     ProductoService, UsuarioService
    │   ├── repository/  ProductoRepository, CategoriaRepository,
    │   │                UsuarioRepository, VentaRepository
    │   └── model/       Producto, Categoria, Usuario,
    │                    Venta, DetalleVenta
    └── resources/
        ├── application.properties
        ├── import.sql
        ├── templates/   index, admin-productos, admin-usuarios
        └── static/      CSS, js, imagenes
```

3.3. Implementación del sistema

El desarrollo se realizó en **Visual Studio Code** con las extensiones Extension Pack for Java y Spring Boot Extension Pack. La compilación y las dependencias se gestionan con **Maven**. El proyecto utiliza **Spring Boot 4.1.1** (con Hibernate ORM 7.4.5 y Apache Tomcat 11.0.24), está configurado para **Java 25** en el pom.xml y se ejecutó con el JDK 26; como base de datos usa **PostgreSQL 18** (versión 18.3) administrada con **pgAdmin 4**. La aplicación se ejecuta desde la terminal con el comando mvnw.cmd spring-boot:run, o desde la clase LinksportBackendApplication, que inicia el servidor Tomcat embebido en el puerto 8080.

Tabla 3

Dependencias principales declaradas en el archivo pom.xml.

Dependencia	Función
spring-boot-starter-webmvc	Controladores MVC y REST, y servidor Tomcat embebido.
spring-boot-starter-thymeleaf	Motor de plantillas para generar el HTML en el servidor.
spring-boot-starter-data-jpa	JPA, Hibernate y repositorios Spring Data.
spring-boot-starter-validation	Bean Validation (Spring Validation) con las anotaciones de restricción.
postgresql (runtime)	Controlador JDBC para conectar con PostgreSQL.
spring-boot-devtools	Reinicio automático durante el desarrollo.

Para ejecutar el sistema se creó en pgAdmin la base de datos **linksport_db**; las tablas no se crean manualmente, pues Hibernate las genera al iniciar la aplicación y el archivo import.sql carga los datos iniciales.

Figura 2

Consola de VS Code mostrando el arranque de Spring Boot 4.1.1, Tomcat en el puerto 8080 y el escaneo de los repositorios Spring Data JPA.



3.4. Implementación del backend con Spring

El backend cumple el criterio de modularidad mediante paquetes y clases independientes dentro del paquete raíz com.linksport.linksport_backend, como se observa en la estructura del proyecto.

a) Paquete service

ProductoService concentra la lógica del catálogo y ya no utiliza colecciones en memoria: recibe por constructor los repositorios de productos, ventas y usuarios, y delega en ellos el acceso a la base de datos. Expone los métodos obtenerTodos(), buscarPorId(), guardar(), actualizar() y eliminar(), además de registrarVentaDesdeWeb(), que persiste las compras del carrito.

Código 2

Fragmento de ProductoService (búsqueda, edición y eliminación).

```
@Service
public class ProductoService {
    private final ProductoRepository productoRepository;
    private final VentaRepository ventaRepository;
    private final UsuarioRepository usuarioRepository;

    public ProductoService(ProductoRepository productoRepository,
            VentaRepository ventaRepository,
            UsuarioRepository usuarioRepository) { ... }
```

```
    public List<Producto> obtenerTodos() {
        return productoRepository.findAll();
    }

    public Optional<Producto> buscarPorId(Long id) {
        return productoRepository.findById(id);
    }

    public Producto actualizar(Long id, Producto datos) {
        Optional<Producto> opt = buscarPorId(id);
        if (opt.isPresent()) {
            Producto p = opt.get();
            p.setNombre(datos.getNombre());
            p.setPrecio(datos.getPrecio());
            p.setImagenSrc(datos.getImagenSrc());
            return productoRepository.save(p);
        }
        return null;
    }

    public boolean eliminar(Long id) {
        if (productoRepository.existsById(id)) {
            productoRepository.deleteById(id);
            return true;
        }
        return false;
    }
}
```

El método **registrarVentaDesdeWeb()** recibe la lista de productos del carrito, crea la cabecera de la venta asociada al usuario, y por cada producto genera un detalle con la cantidad y el precio unitario tomado de la base de datos. Gracias a la cascada definida en la relación, una sola llamada a save() guarda la venta y todos sus detalles.

Código 3

Fragmento de registrarVentaDesdeWeb en ProductoService.

```
public void registrarVentaDesdeWeb(List<Producto> carrito) {
    if (carrito == null || carrito.isEmpty()) return;

    var usuarioOpt = usuarioRepository.findById(1L);
    if (usuarioOpt.isEmpty()) return;

    Venta nuevaVenta = new Venta();
    nuevaVenta.setUsuario(usuarioOpt.get());

    double total = 0;
    for (Producto p : carrito) {
        total += p.getPrecio() * p.getCantidad();
    }
    nuevaVenta.setTotalPagar(total);

    for (Producto p : carrito) {
        var real = productoRepository.findById(p.getId());
        if (real.isPresent()) {
            DetalleVenta d = new DetalleVenta();
            d.setVenta(nuevaVenta);
            d.setProducto(real.get());
            d.setCantidad(p.getCantidad());
            d.setPrecioUnitario(real.get().getPrecio());
            nuevaVenta.getDetalles().add(d);
        }
    }
    ventaRepository.save(nuevaVenta);   // guarda venta + detalles
}
```

UsuarioService expone obtenerTodos(), registrar() y buscarPorCorreo(). En esta versión conserva la colección ArrayList del Avance 2 para el registro de cuentas; la entidad Usuario ya está mapeada a la tabla usuarios y es la que utiliza el registro de ventas. La migración del servicio al repositorio de usuarios se contempla como mejora (véanse las conclusiones).

b) Paquete controller

ProductoController y **UsuarioController** son controladores REST que reciben su servicio por constructor. **HomeController** es un controlador de vistas que atiende las rutas / y /ver-productos, filtra los productos por categoría y por la marca de destacado, y procesa el formulario del panel (/admin/productos/guardar). Las operaciones del CRUD se corresponden con los endpoints de la Tabla 4.

Tabla 4

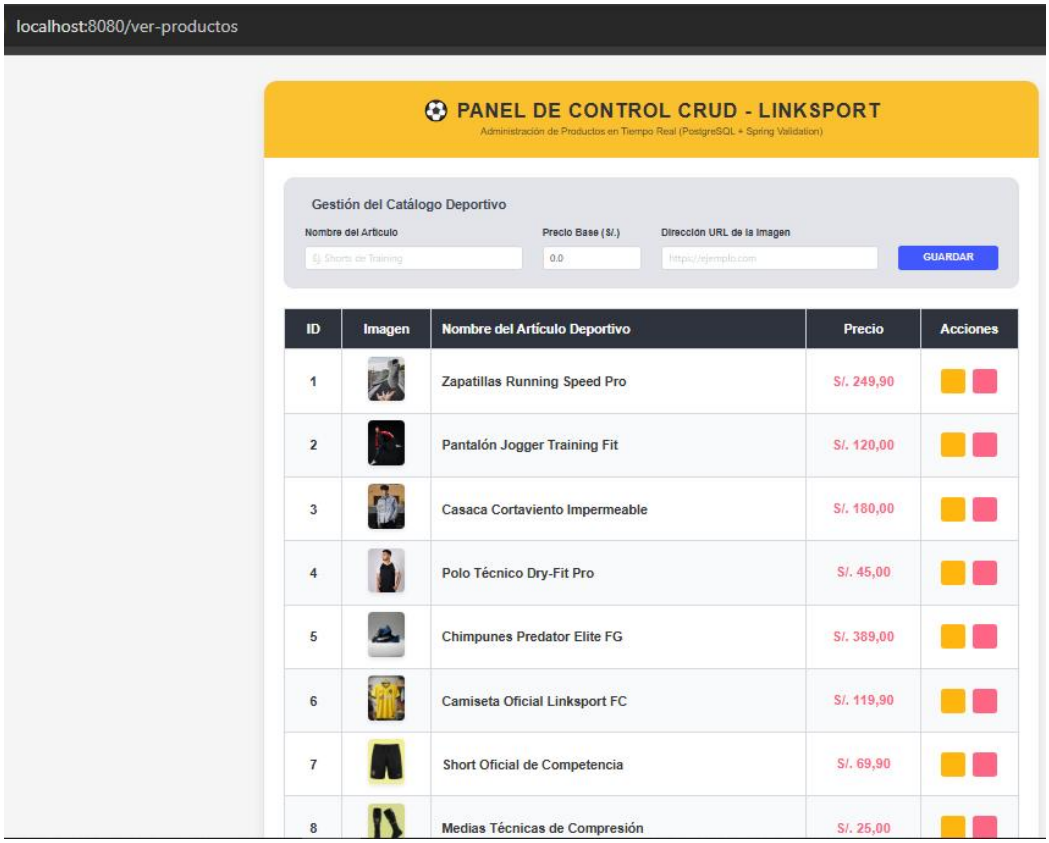
Endpoints que implementan el CRUD y el registro de ventas.

Operación	Método y ruta	Respuesta	Descripción
Mostrar	GET /api/productos	200	Lista todos los productos de la base de datos.
Buscar	GET /api/productos/{id}	200 / 404	Busca un producto por su identificador.
Ingresar	POST /api/productos	200 / 400	Registra un producto; responde 400 si los datos no son válidos o el nombre está repetido en la categoría.
Editar	PUT /api/productos/{id}	200 / 400 / 404	Actualiza nombre, precio e imagen, validando los datos.
Eliminar	DELETE /api/productos/{id}	200 / 404	Elimina el producto indicado.
Registrar venta	POST /api/productos/procesar-pago	200	Guarda la venta y sus detalles a partir del carrito.
Registrar usuario	POST /api/usuarios/registro	200 / 400	Registra una cuenta; rechaza correos duplicados.
Mostrar usuarios	GET /api/usuarios	200	Lista las cuentas registradas.
Panel web	GET /ver-productos	200	Muestra el formulario y la tabla de inventario (acepta ?id= para editar).
Guardar desde panel	POST /admin/productos/guardar	302	Guarda el producto y redirige a /ver-productos.

Las operaciones REST se verificaron con el archivo pruebas_crud.http, ejecutado desde VS Code mediante la extensión REST Client.

Figura 3

Panel de productos (/ver-productos) con el formulario y la tabla de inventario cargada desde PostgreSQL.



3.5. Implementación con JPA - Hibernate

a) Configuración de la conexión

La conexión con PostgreSQL y el comportamiento de Hibernate se definen en el archivo application.properties. La propiedad ddl-auto con el valor create-drop indica a Hibernate que cree las tablas a partir de las entidades al iniciar y las elimine al detener la aplicación; la propiedad show-sql permite observar en consola las sentencias SQL generadas.

Código 4

Configuración de application.properties (la contraseña se omite).

```
spring.datasource.url=jdbc:postgresql://localhost:5432/linksport_db
spring.datasource.username=postgres
spring.datasource.password=********
spring.datasource.driver-class-name=org.postgresql.Driver

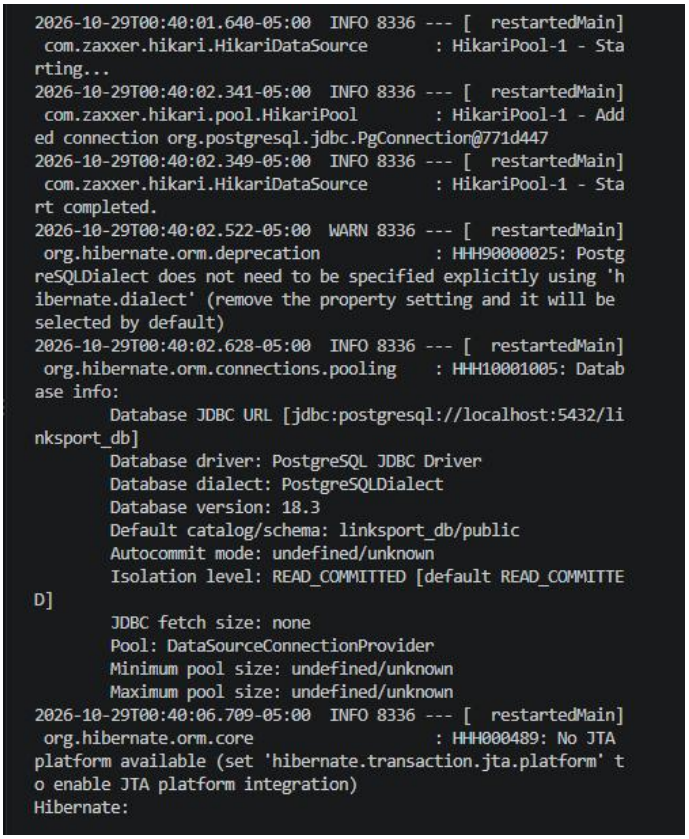
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false

spring.sql.init.mode=always
spring.jpa.defer-datasource-initialization=true
```

Al iniciar, Spring Boot crea un pool de conexiones HikariCP hacia la base linksport_db en PostgreSQL 18.3 y Hibernate inicializa la unidad de persistencia, como se observa en la Figura 4.

Figura 4

Conexión de la aplicación con PostgreSQL mediante HikariCP y Hibernate ORM 7.4.5.



b) Entidades y relaciones

Se definieron cinco entidades, cada una anotada con @Entity y @Table, con clave primaria autogenerada (@Id y @GeneratedValue con estrategia IDENTITY) y restricciones de columna con @Column. La entidad Producto, por ejemplo, concentra tanto el mapeo como las reglas de validación.

Código 5

Fragmento de la entidad Producto.

```
@Entity
@Table(name = "productos")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String nombre;

    @Column(nullable = false)
    private double precio;

    @Column(nullable = false)
    private boolean destacado = false;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "categoria_id", nullable = true)
    private Categoria categoria;
    ...
}
```

Las entidades se relacionan mediante asociaciones uno a muchos, que en la base de datos se materializan como claves foráneas. La Figura 5 muestra el modelo relacional resultante y la Tabla 5 detalla cada relación.

Figura 5

Modelo relacional de LINKSPORT generado por JPA e Hibernate.

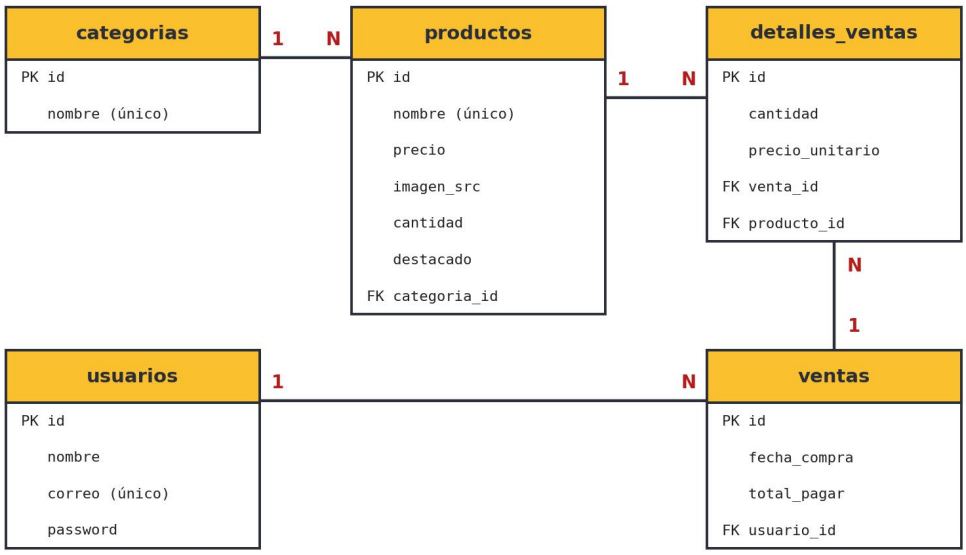


Tabla 5

Relaciones entre las entidades del modelo.

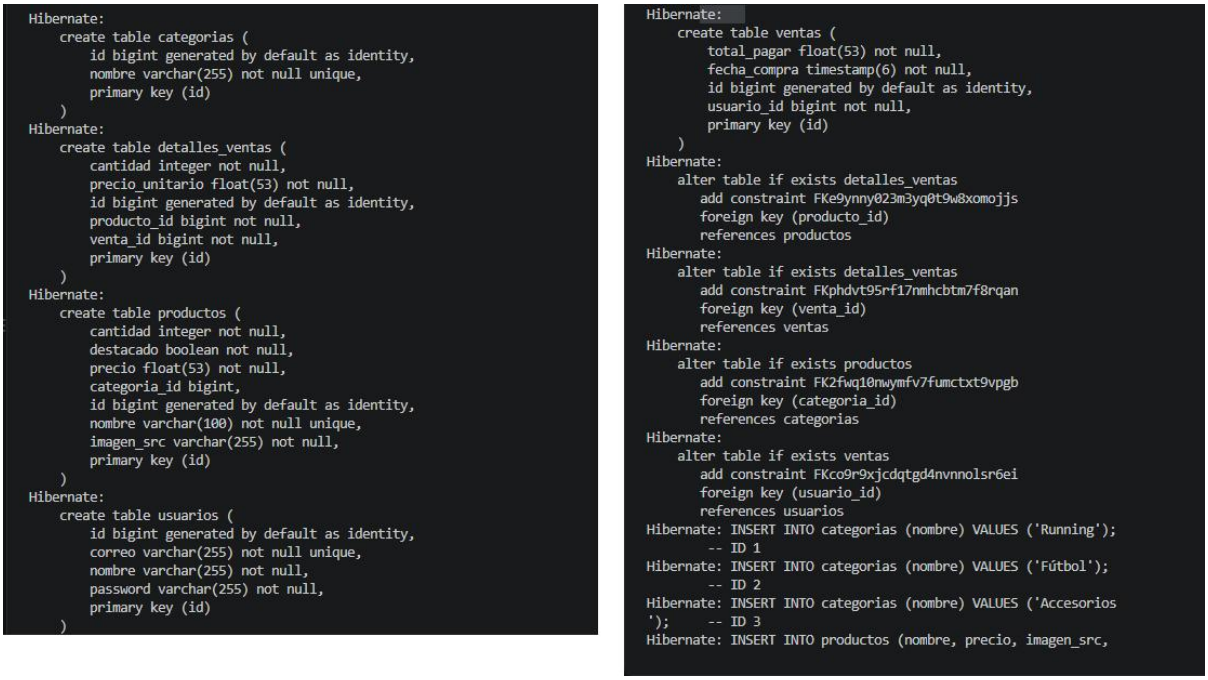
Relación	Cardinalidad	Mapeo JPA	Clave foránea

Relación	Cardinalidad	Mapeo JPA	Clave foránea
Categoría - Producto	1 a N	@OneToMany(mappedBy = "categoria", cascade = ALL) en Categoría; @ManyToOne(LAZY) en Producto	productos.categoria_id (opcional)
Usuario - Venta	1 a N	@ManyToOne(LAZY) en Venta	ventas.usuario_id (obligatoria)
Venta - DetalleVenta	1 a N	@OneToMany(mappedBy = "venta", cascade = ALL, orphanRemoval = true) en Venta; @ManyToOne(LAZY) en DetalleVenta	detalles_ventas.venta_id (obligatoria)
Producto - DetalleVenta	1 a N	@ManyToOne(LAZY) en DetalleVenta	detalles_ventas.producto_id (obligatoria)

Con la propiedad show-sql activa, la consola muestra las sentencias DDL que Hibernate ejecuta al arrancar: la creación de las cinco tablas y las restricciones de clave foránea (alter table ... add constraint ... foreign key) que materializan las relaciones (Figura 6).

Figura 6

Sentencias DDL generadas por Hibernate: tablas y claves foráneas.



```
Hibernate:
    create table categorias (
        id bigint generated by default as identity,
        nombre varchar(255) not null unique,
        primary key (id)
    )
Hibernate:
    create table detalles_ventas (
        cantidad integer not null,
        precio_unitario float(53) not null,
        id bigint generated by default as identity,
        producto_id bigint not null,
        venta_id bigint not null,
        primary key (id)
    )
Hibernate:
    create table productos (
        cantidad integer not null,
        destacado boolean not null,
        precio float(53) not null,
        categoria_id bigint,
        id bigint generated by default as identity,
        nombre varchar(100) not null unique,
        imagen_src varchar(255) not null,
        primary key (id)
    )
Hibernate:
    create table usuarios (
        id bigint generated by default as identity,
        correo varchar(255) not null unique,
        nombre varchar(255) not null,
        password varchar(255) not null,
        primary key (id)
    )
```

```
Hibernate:
    create table ventas (
        total_pagar float(53) not null,
        fecha_compra timestamp(6) not null,
        id bigint generated by default as identity,
        usuario_id bigint not null,
        primary key (id)
    )
Hibernate:
    alter table if exists detalles_ventas
       add constraint FKe9ynny023m3yq0t9w8xomojjs
       foreign key (producto_id)
       references productos
Hibernate:
    alter table if exists detalles_ventas
       add constraint FKphdvt95rf17nmhcbtm7f8rqan
       foreign key (venta_id)
       references ventas
Hibernate:
    alter table if exists productos
       add constraint FK2fwq10nwymfv7fumctxt9vpgb
       foreign key (categoria_id)
       references categorias
Hibernate:
    alter table if exists ventas
       add constraint FKco9r9xjcdqtgd4nvnnolsr6ei
       foreign key (usuario_id)
       references usuarios
Hibernate: INSERT INTO categorias (nombre) VALUES ('Running');
        -- ID 1
Hibernate: INSERT INTO categorias (nombre) VALUES ('Fútbol');
        -- ID 2
Hibernate: INSERT INTO categorias (nombre) VALUES ('Accesorios
');      -- ID 3
Hibernate: INSERT INTO productos (nombre, precio, imagen_src,
```

Código 6

Relación entre Venta y DetalleVenta (cabecera y detalle).

```
// En la entidad Venta
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "usuario_id", nullable = false)
private Usuario usuario;

@OneToMany(mappedBy = "venta", cascade = CascadeType.ALL,
           orphanRemoval = true)
private List<DetalleVenta> detalles = new ArrayList<>();

// En la entidad DetalleVenta
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "venta_id", nullable = false)
private Venta venta;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "producto_id", nullable = false)
private Producto producto;
```

c) Repositorios Spring Data JPA

El acceso a datos se resuelve con interfaces que extienden JpaRepository, por lo que no es necesario escribir sentencias SQL para el CRUD. En UsuarioRepository se declara además una consulta derivada para buscar por correo.

Código 7

Repositorios del paquete repository.

```
public interface ProductoRepository extends JpaRepository<Producto, Long> { }

public interface CategoriaRepository extends JpaRepository<Categoria, Long> { }

public interface VentaRepository extends JpaRepository<Venta, Long> { }

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByCorreo(String correo);
}
```

d) Carga inicial de datos y verificación en PostgreSQL

El archivo import.sql registra las tres categorías (Running, Fútbol y Accesorios), doce productos asociados a ellas, con la marca de destacado en los más vendidos, y un usuario de prueba. Los datos se cargan automáticamente en cada arranque, de modo que el catálogo está disponible en cuanto se inicia la aplicación.

Código 8

Fragmento de import.sql.

```
INSERT INTO categorias (nombre) VALUES ('Running');
INSERT INTO categorias (nombre) VALUES ('Fútbol');
INSERT INTO categorias (nombre) VALUES ('Accesorios');

INSERT INTO productos (nombre, precio, imagen_src, cantidad,
                       categoria_id, destacado)
VALUES ('Zapatillas Running Speed Pro', 249.90,
        'imagenes/producto1.jpg', 1, 1, true);

INSERT INTO usuarios (nombre, correo, password)
VALUES ('Juan Pérez Linksport', 'juan@linksport.com', '123456');
```

En pgAdmin se comprobó que Hibernate creó en la base de datos linksport_db las cinco tablas del modelo: categorias, detalles_ventas, productos, usuarios y ventas (Figura 7). La captura muestra además el script SQL de la tabla categorias, con su clave primaria autogenerada y la restricción de unicidad sobre el nombre.

Figura 7

Tablas generadas por Hibernate en la base de datos linksport_db (pgAdmin 4).

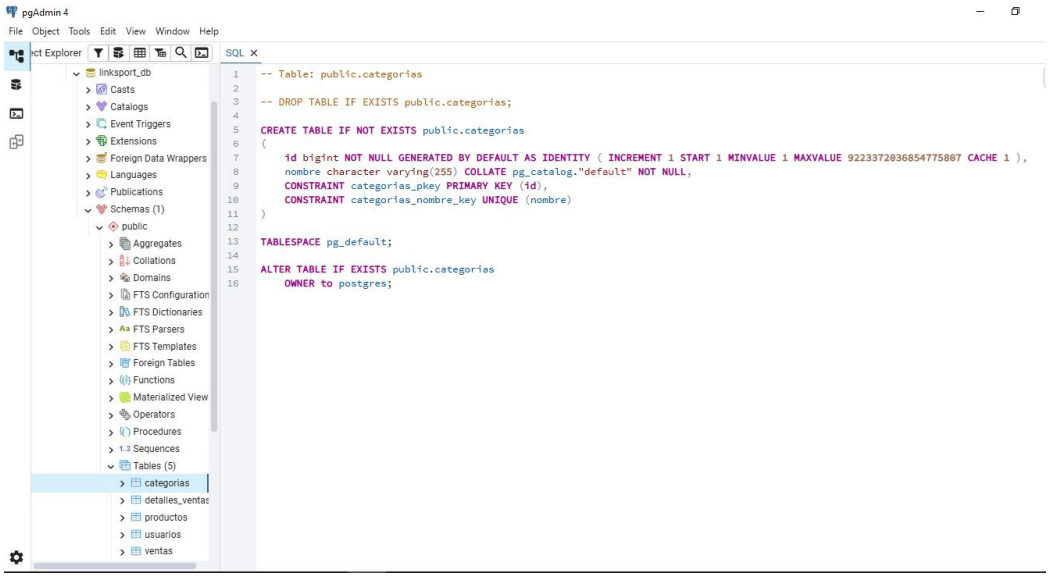
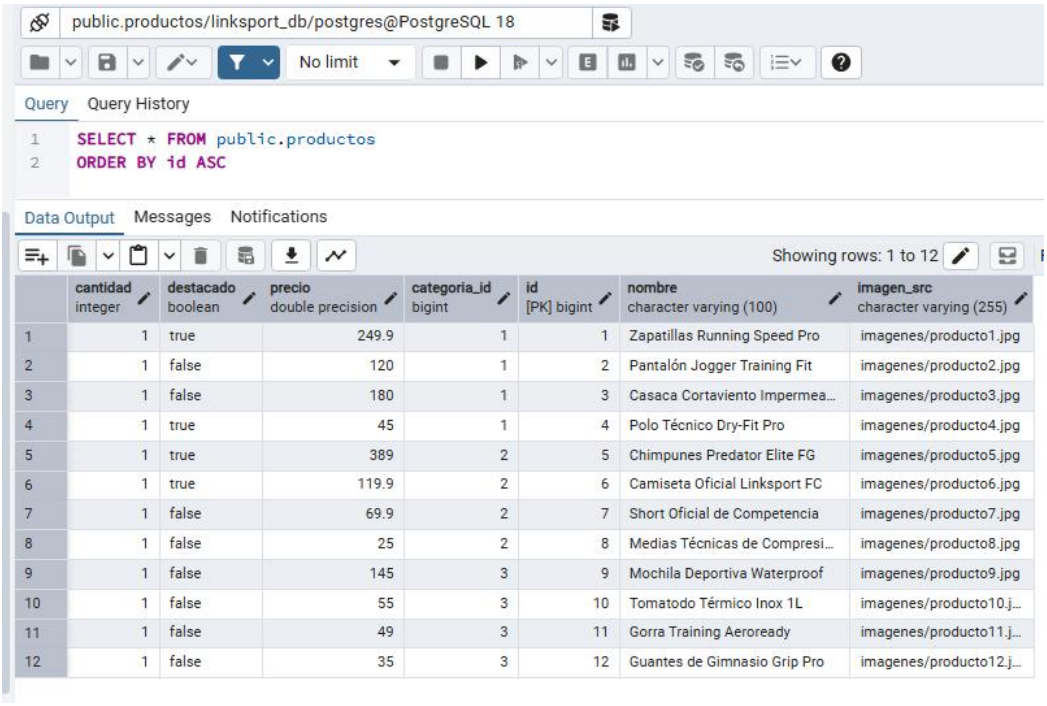


Figura 8

Contenido de la tabla productos en pgAdmin 4: 12 productos con su categoría y marca de destacado.



	cantidad integer	destacado boolean	precio double precision	categoria_id bigint	id [PK] bigint	nombre character varying (100)	imagen_src character varying (255)
1	1	true	249.9	1	1	Zapatillas Running Speed Pro	imagenes/producto1.jpg
2	1	false	120	1	2	Pantalón Jogger Training Fit	imagenes/producto2.jpg
3	1	false	180	1	3	Casaca Cortaviento Impermea...	imagenes/producto3.jpg
4	1	true	45	1	4	Polo Técnico Dry-Fit Pro	imagenes/producto4.jpg
5	1	true	389	2	5	Chimpunes Predator Elite FG	imagenes/producto5.jpg
6	1	true	119.9	2	6	Camiseta Oficial Linksport FC	imagenes/producto6.jpg
7	1	false	69.9	2	7	Short Oficial de Competencia	imagenes/producto7.jpg
8	1	false	25	2	8	Medias Técnicas de Compresi...	imagenes/producto8.jpg
9	1	false	145	3	9	Mochila Deportiva Waterproof	imagenes/producto9.jpg
10	1	false	55	3	10	Tomatodo Térmico Inox 1L	imagenes/producto10.j...
11	1	false	49	3	11	Gorra Training Aeroready	imagenes/producto11.j...
12	1	false	35	3	12	Guantes de Gimnasio Grip Pro	imagenes/producto12.j...

Para comprobar el registro de ventas, se agregaron al carrito dos productos (Polo Técnico Dry-Fit Pro y Medias Técnicas de Compresión) con un subtotal de S/. 70.00 y se presionó “Proceder al pago” (Figura 9). Como resultado, el sistema insertó una nueva fila en la tabla ventas con total_pagar igual a 70, la fecha de compra y el usuario asociado (Figura 10); las líneas de cada compra se guardan en la tabla detalles_ventas, relacionadas con su venta y su producto mediante las claves foráneas venta_id y producto_id (Figura 11).

Figura 9

Carrito lateral con dos productos y un subtotal de S/. 70.00 antes de proceder al pago.

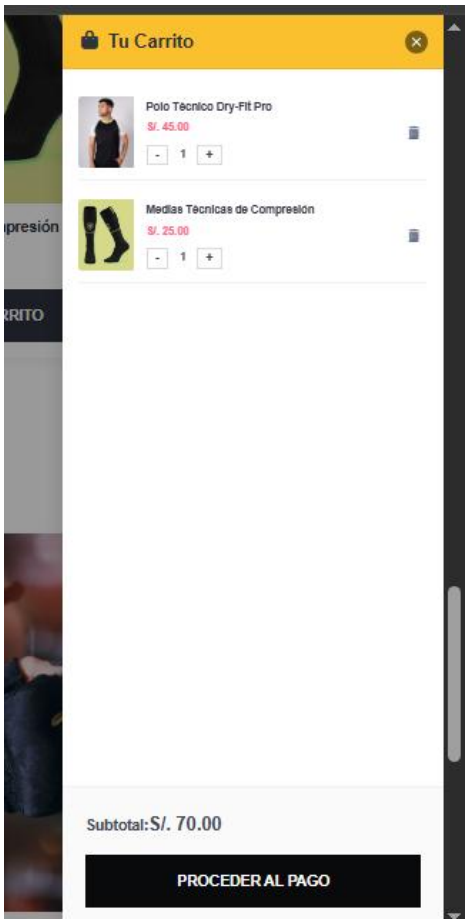
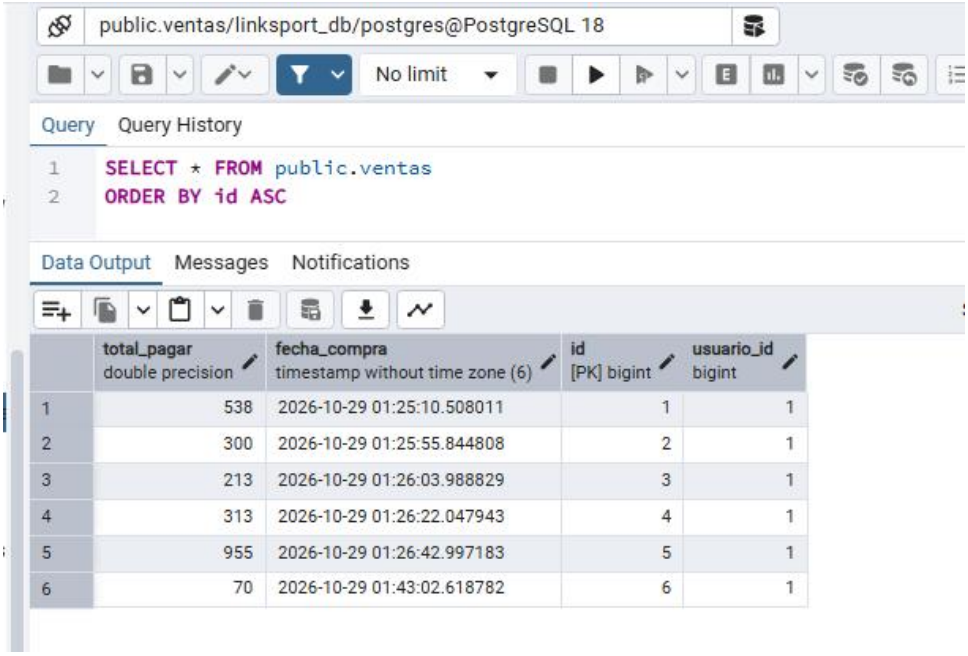


Figura 10

Contenido de la tabla ventas en pgAdmin 4 tras registrar la compra (venta 6 por S/. 70.00).



	total_pagar double precision	fecha_compra timestamp without time zone (6)	id [PK] bigint	usuario_id bigint
1	538	2026-10-29 01:25:10.508011	1	1
2	300	2026-10-29 01:25:55.844808	2	1
3	213	2026-10-29 01:26:03.988829	3	1
4	313	2026-10-29 01:26:22.047943	4	1
5	955	2026-10-29 01:26:42.997183	5	1
6	70	2026-10-29 01:43:02.618782	6	1

3.6. Implementación de Spring Validation

La validación de datos se implementó en tres puntos: las anotaciones de restricción en las clases del modelo, la activación en los controladores y la presentación de los mensajes en las vistas Thymeleaf. Se incorporó la dependencia spring-boot-starter-validation en el pom.xml.

a) Anotaciones en las clases del modelo

Las restricciones se declaran directamente sobre los atributos de las entidades, con mensajes en español orientados al usuario. En esta versión, las mismas clases del modelo se usan como objeto de entrada del formulario web y de la API REST.

Tabla 6

Anotaciones de validación por entidad.

Entidad	Atributo	Anotaciones y regla
Producto	nombre	@NotBlank (obligatorio) y @Size(min = 3, max = 100).
Producto	precio	@Positive (mayor a cero) y @DecimalMin("1.00") (mínimo S/. 1.00).
Producto	imagenSrc	@NotBlank (la dirección de la imagen es obligatoria).
Usuario	nombre	@NotBlank.
Usuario	correo	@NotBlank y @Email (formato de correo válido).
Usuario	password	@NotBlank y @Size(min = 6).

Código 9

Anotaciones de validación en la entidad Producto.

```
@NotBlank(message = "El nombre del artículo deportivo es obligatorio.")
@Size(min = 3, max = 100,
      message = "El nombre debe tener entre 3 y 100 caracteres.")
@Column(nullable = false, unique = true)
private String nombre;

@Positive(message = "El precio debe ser un monto mayor a cero.")
@DecimalMin(value = "1.00",
            message = "El precio mínimo permitido es de S/. 1.00")
@Column(nullable = false)
private double precio;

@NotBlank(message = "La dirección URL de la imagen es obligatoria.")
@Column(nullable = false)
private String imagenSrc;
```

b) Activación en los controladores

En la API REST, ProductoController activa la validación con @Valid sobre el cuerpo de la petición en los métodos POST y PUT; si los datos no cumplen las restricciones, Spring responde automáticamente con el código 400 sin ejecutar la lógica del método. Además, se

aplica una regla de negocio propia: no se permite registrar un producto cuyo nombre ya exista dentro de la misma categoría.

Código 10

Activación de @Valid en ProductoController.

```
@PostMapping
public ResponseEntity<?> crearProducto(
        @Valid @RequestBody Producto producto) {

    boolean duplicado = productoService.obtenerTodos().stream()
        .anyMatch(p -> p.getNombre().equalsIgnoreCase(producto.getNombre())
            && p.getCategoria() != null && producto.getCategoria() != null
            && p.getCategoria().getId()
                 .equals(producto.getCategoria().getId()));

    if (duplicado) {
        return ResponseEntity.badRequest().body(
            "Error: Este producto ya está registrado en esta sección.");
    }
    return ResponseEntity.ok(productoService.guardar(producto));
}

@PutMapping("/{id}")
public ResponseEntity<Producto> modificarProducto(
        @PathVariable Long id, @Valid @RequestBody Producto producto) { ... }
```

En el panel web, HomeController recibe el formulario con @Valid y @ModelAttribute, y un objeto BindingResult que almacena los errores. Si existen errores, no se guarda el producto y se vuelve a mostrar el formulario; si no los hay, se guarda y se redirige al listado. Con result.rejectValue() también se registra el error de duplicado como un error del campo nombre.

Código 11

Validación en HomeController con BindingResult.

```
@PostMapping("/admin/productos/guardar")
public String guardarProductoDesdeWeb(
        @Valid @ModelAttribute("producto") Producto producto,
        BindingResult result, Model model) {

    if (producto.getId() == null && duplicadoEnMismaCategoria(producto)) {
        result.rejectValue("nombre", "error.producto",
            "Ya existe un artículo registrado en esta sección con este nombre.");
    }

    if (result.hasErrors()) {
        model.addAttribute("productos", productoService.obtenerTodos());
        return "admin-productos";          // vuelve al formulario con errores
    }

    productoService.guardar(producto);
    return "redirect:/ver-productos";
}
```

c) Condiciones en el front end para mostrar los errores

La plantilla admin-productos.html enlaza cada campo del formulario con el objeto del modelo mediante th:field y, debajo de cada uno, muestra el mensaje de error solo cuando el

campo no cumple la restricción, usando #fields.hasErrors() y th:errors. El estilo is-danger de Bulma resalta el mensaje en rojo.

Código 12

Condición para mostrar el error de validación en el formulario (Thymeleaf).

```
<form th:action="@{/admin/productos/guardar}"
      th:object="${producto}" method="POST">
  <input type="hidden" th:field="*{id}">

  <input class="input is-small" type="text" th:field="*{nombre}">
  <p class="help is-danger has-text-weight-semibold mt-1"
     th:if="${#fields.hasErrors('nombre')}"
     th:errors="*{nombre}">Error</p>

  <input class="input is-small" type="number" step="0.01"
         th:field="*{precio}">
  <p class="help is-danger" th:if="${#fields.hasErrors('precio')}"
     th:errors="*{precio}">Error</p>
  ...
</form>
```

Figura 12

Formulario del panel mostrando los mensajes de validación cuando se envía con el nombre vacío, un precio menor a S/. 1.00 y sin imagen.



3.7. Framework front-end (Bulma)

Como alternativa a Bootstrap se utilizó **Bulma 1.0.4**, un framework CSS similar basado en Flexbox, cargado desde una red de distribución de contenidos (CDN), sobre plantillas Thymeleaf con HTML5 semántico (header, nav, main, section y footer). Los componentes principales son:

- **Menú de navegación responsivo:** componente navbar con botón navbar-burger, que en pantallas pequeñas despliega el menú colapsable.
- **Catálogo por categorías:** contenedor columns is-multiline con una column is-3 por producto, generada con th:each; se muestran cuatro productos por fila en pantallas medianas y grandes, y una sola columna en móviles. Cada producto se presenta en un componente card.

- **Tabla de inventario y formulario:** table is-bordered is-striped is-hoverable is-fullwidth en el panel de administración, con botones de edición y eliminación, y un formulario field/control/input para registrar o editar.
- **Ventanas modales y carrito lateral:** el inicio de sesión, el registro y la confirmación de compra usan el componente modal; el carrito es un panel lateral propio con el subtotal en tiempo real.

Dado que Bulma es exclusivamente CSS, el comportamiento interactivo se programó en el archivo script.js: carrusel de imágenes, carrito de compras, apertura y cierre de modales y envío de la compra al servidor. Al presionar “Proceder al pago”, el script envía el contenido del carrito mediante fetch al endpoint /api/productos/procesar-pago, y si la respuesta es exitosa muestra la confirmación y vacía el carrito.

Código 13

Envío del carrito al backend desde script.js.

```
fetch('/api/productos/procesar-pago', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(mapeoProductos)
})
.then(res => {
    if (!res.ok) throw new Error("Error procesando pago");
    window.cerrarPanelCarrito();
    modalCompra.classList.add('is-active');
    productosEnCarrito = [];
    actualizarPantallaCarrito();
})
.catch(err => alert("Error al registrar la transacción..."));
```

Figura 13

Sección “Lo más vendido” y colección Running generadas con Thymeleaf desde la base de datos.

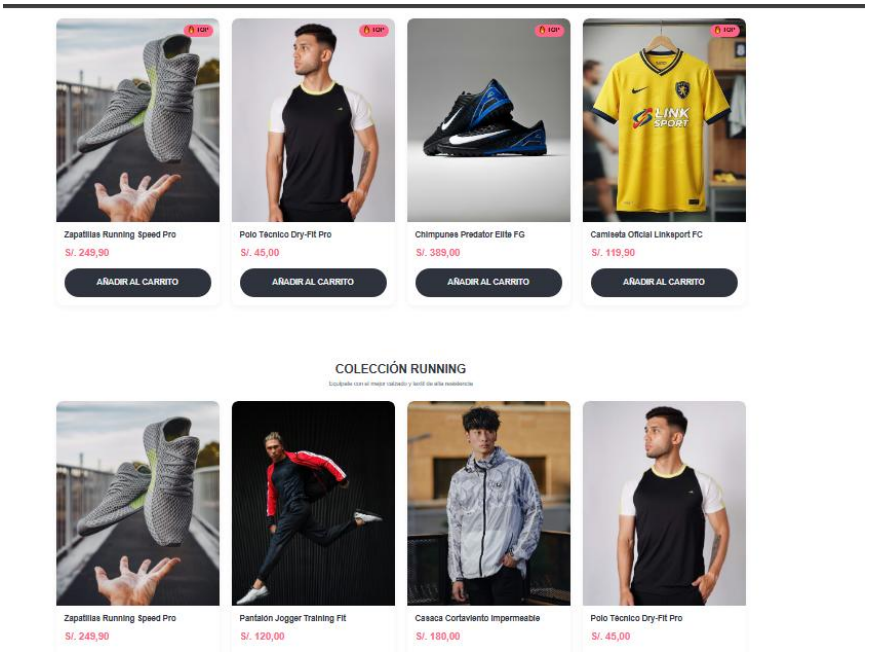


Figura 14

Colecciones Universo Fútbol y Accesorios Deportivos.

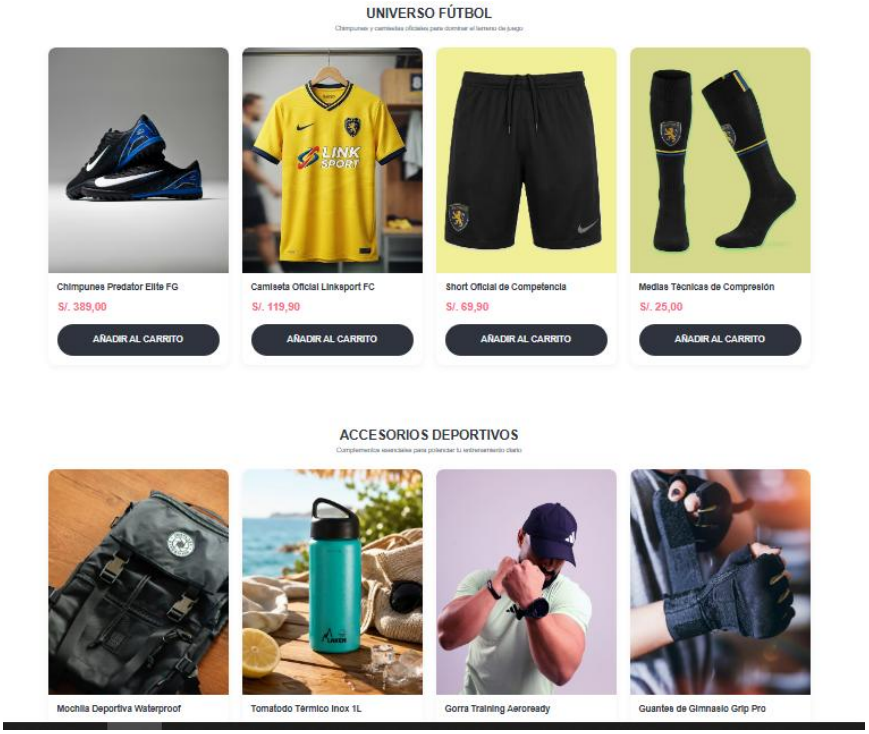


Figura 15

Ventana modal de inicio de sesión con el cupón de bienvenida.

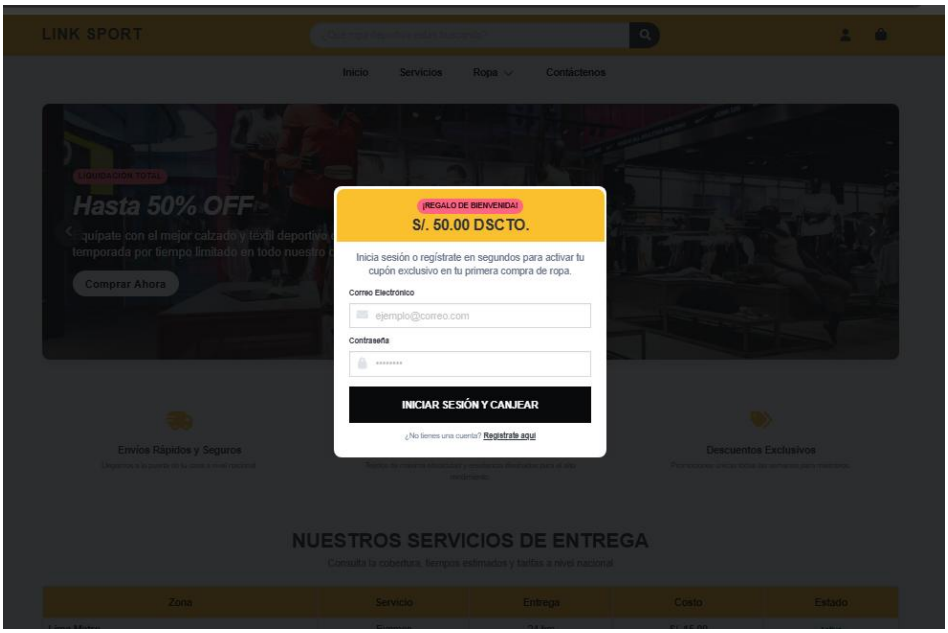
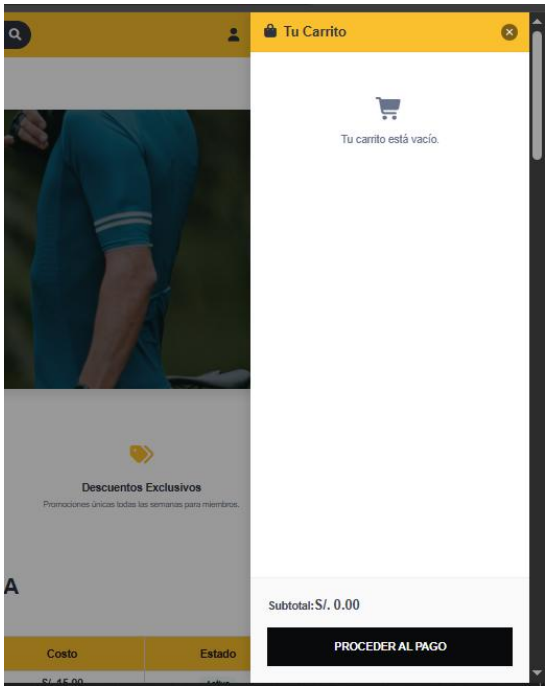


Figura 16

Carrito lateral con el subtotal y el botón “Proceder al pago”.



CONCLUSIONES

1. La integración de JPA e Hibernate con PostgreSQL permitió reemplazar el almacenamiento en memoria del catálogo por tablas relacionales, cargadas automáticamente desde import.sql al iniciar la aplicación. La configuración create-drop recrea el esquema en cada arranque, por lo que es adecuada para desarrollo; los datos ingresados durante la ejecución no se conservan entre reinicios y, para un entorno de producción, deberá cambiarse por un modo que preserve la información.
2. El modelo de cinco entidades (Categoría, Producto, Usuario, Venta y DetalleVenta), con relaciones uno a muchos y claves foráneas, permite organizar el catálogo por secciones y registrar cada compra como una cabecera con sus detalles, guardados de forma conjunta gracias a la cascada.
3. Spring Validation, con anotaciones en las entidades, activación mediante @Valid y BindingResult en los controladores, y el uso de th:errors en las vistas, evita que ingresen al sistema nombres vacíos, precios inválidos o productos sin imagen, y comunica el error al usuario de forma clara.
4. La arquitectura por capas con repositorios Spring Data permitió sustituir la persistencia sin modificar las vistas, tal como se anticipó en el Avance 2, y mantiene el código organizado en paquetes de responsabilidad única.
5. Bulma y Thymeleaf permitieron construir una interfaz responsiva cuyo catálogo se actualiza según los datos de la base, y un carrito que registra las compras en el servidor.
6. Como limitaciones y trabajo futuro se identifican: (a) unificar el registro de usuarios en la base de datos mediante UsuarioRepository e implementar un inicio de sesión real, pues actualmente el acceso del front end es una simulación y las ventas se asignan a un usuario de prueba; (b) cifrar las contraseñas y proteger los endpoints con Spring Security; (c) identificar los productos del carrito por su identificador de base de datos y calcular el total de la venta en el servidor con los precios almacenados; (d) separar las reglas de validación en objetos DTO independientes de las entidades; y (e) reemplazar create-drop por un esquema persistente con control de versiones de la base de datos.

REFERENCIAS

Bauer, C., King, G., & Gregory, G. (2015). *Java persistence with Hibernate* (2.ª ed.). Manning Publications.

Bulma. (2025). *Bulma documentation*. https://bulma.io/documentation/

CAPECE. (2023). *Reporte oficial de la industria Ecommerce en Perú 2022-2023*. Cámara Peruana de Comercio Electrónico. https://www.capece.org.pe/congreso-observatorio-ecommerce/

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Hibernate. (s. f.). *Hibernate ORM documentation*. https://hibernate.org/orm/documentation/

Jakarta EE. (s. f.). *Jakarta Bean Validation*. https://jakarta.ee/specifications/bean-validation/

Marcotte, E. (2011). *Responsive web design*. A Book Apart.

PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Spring. (s. f.-a). *Spring Boot reference documentation*. https://docs.spring.io/spring-boot/

Spring. (s. f.-b). *Spring Data JPA*. https://spring.io/projects/spring-data-jpa

Thymeleaf. (2025). *Using Thymeleaf*. https://www.thymeleaf.org/documentation.html

Walls, C. (2022). *Spring in action* (6.ª ed.). Manning Publications.